

it holds for n , and we prove that it holds for $n + 1$. We have

$$\begin{aligned}\sum_{k=1}^{n+1} k &= \sum_{k=1}^n k + (n+1) \\ &= \frac{1}{2}n(n+1) + (n+1) \\ &= \frac{1}{2}(n+1)(n+2) .\end{aligned}$$

You don't always need to guess the exact value of a summation in order to use mathematical induction. Instead, you can use induction to prove a bound on a summation. As an example, let us prove that the geometric series $\sum_{k=0}^n 3^k$ is $O(3^n)$. More specifically, let us prove that $\sum_{k=0}^n 3^k \leq c3^n$ for some constant c . For the initial condition $n = 0$, we have $\sum_{k=0}^0 3^k = 1 \leq c \cdot 1$ as long as $c \geq 1$. Assuming that the bound holds for n , let us prove that it holds for $n + 1$. We have

$$\begin{aligned}\sum_{k=0}^{n+1} 3^k &= \sum_{k=0}^n 3^k + 3^{n+1} \\ &\leq c3^n + 3^{n+1} \quad (\text{by the inductive hypothesis}) \\ &= \left(\frac{1}{3} + \frac{1}{c}\right) c3^{n+1} \\ &\leq c3^{n+1}\end{aligned}$$

as long as $(1/3 + 1/c) \leq 1$ or, equivalently, $c \geq 3/2$. Thus, $\sum_{k=0}^n 3^k = O(3^n)$, as we wished to show.

We have to be careful when we use asymptotic notation to prove bounds by induction. Consider the following fallacious proof that $\sum_{k=1}^n k = O(n)$. Certainly, $\sum_{k=1}^1 k = O(1)$. Assuming that the bound holds for n , we now prove it for $n + 1$:

$$\begin{aligned}\sum_{k=1}^{n+1} k &= \sum_{k=1}^n k + (n+1) \\ &= O(n) + (n+1) \quad \Leftarrow \text{wrong!!} \\ &= O(n+1) .\end{aligned}$$

The bug in the argument is that the “constant” hidden by the “big-oh” grows with n and thus is not constant. We have not shown that the same constant works for *all* n .

Bounding the terms

We can sometimes obtain a good upper bound on a series by bounding each term of the series, and it often suffices to use the largest term to bound the others. For

example, a quick upper bound on the arithmetic series (A.1) is

$$\begin{aligned}\sum_{k=1}^n k &\leq \sum_{k=1}^n n \\ &= n^2 .\end{aligned}$$

In general, for a series $\sum_{k=1}^n a_k$, if we let $a_{\max} = \max_{1 \leq k \leq n} a_k$, then

$$\sum_{k=1}^n a_k \leq n \cdot a_{\max} .$$

The technique of bounding each term in a series by the largest term is a weak method when the series can in fact be bounded by a geometric series. Given the series $\sum_{k=0}^n a_k$, suppose that $a_{k+1}/a_k \leq r$ for all $k \geq 0$, where $0 < r < 1$ is a constant. We can bound the sum by an infinite decreasing geometric series, since $a_k \leq a_0 r^k$, and thus

$$\begin{aligned}\sum_{k=0}^n a_k &\leq \sum_{k=0}^{\infty} a_0 r^k \\ &= a_0 \sum_{k=0}^{\infty} r^k \\ &= a_0 \frac{1}{1-r} .\end{aligned}$$

We can apply this method to bound the summation $\sum_{k=1}^{\infty} (k/3^k)$. In order to start the summation at $k = 0$, we rewrite it as $\sum_{k=0}^{\infty} ((k+1)/3^{k+1})$. The first term (a_0) is $1/3$, and the ratio (r) of consecutive terms is

$$\begin{aligned}\frac{(k+2)/3^{k+2}}{(k+1)/3^{k+1}} &= \frac{1}{3} \cdot \frac{k+2}{k+1} \\ &\leq \frac{2}{3}\end{aligned}$$

for all $k \geq 0$. Thus, we have

$$\begin{aligned}\sum_{k=1}^{\infty} \frac{k}{3^k} &= \sum_{k=0}^{\infty} \frac{k+1}{3^{k+1}} \\ &\leq \frac{1}{3} \cdot \frac{1}{1-2/3} \\ &= 1 .\end{aligned}$$

A common bug in applying this method is to show that the ratio of consecutive terms is less than 1 and then to assume that the summation is bounded by a geometric series. An example is the infinite harmonic series, which diverges since

$$\begin{aligned}\sum_{k=1}^{\infty} \frac{1}{k} &= \lim_{n \rightarrow \infty} \sum_{k=1}^n \frac{1}{k} \\ &= \lim_{n \rightarrow \infty} \Theta(\lg n) \\ &= \infty.\end{aligned}$$

The ratio of the $(k+1)$ st and k th terms in this series is $k/(k+1) < 1$, but the series is not bounded by a decreasing geometric series. To bound a series by a geometric series, we must show that there is an $r < 1$, which is a *constant*, such that the ratio of all pairs of consecutive terms never exceeds r . In the harmonic series, no such r exists because the ratio becomes arbitrarily close to 1.

Splitting summations

One way to obtain bounds on a difficult summation is to express the series as the sum of two or more series by partitioning the range of the index and then to bound each of the resulting series. For example, suppose we try to find a lower bound on the arithmetic series $\sum_{k=1}^n k$, which we have already seen has an upper bound of n^2 . We might attempt to bound each term in the summation by the smallest term, but since that term is 1, we get a lower bound of n for the summation—far off from our upper bound of n^2 .

We can obtain a better lower bound by first splitting the summation. Assume for convenience that n is even. We have

$$\begin{aligned}\sum_{k=1}^n k &= \sum_{k=1}^{n/2} k + \sum_{k=n/2+1}^n k \\ &\geq \sum_{k=1}^{n/2} 0 + \sum_{k=n/2+1}^n (n/2) \\ &= (n/2)^2 \\ &= \Omega(n^2),\end{aligned}$$

which is an asymptotically tight bound, since $\sum_{k=1}^n k = O(n^2)$.

For a summation arising from the analysis of an algorithm, we can often split the summation and ignore a constant number of the initial terms. Generally, this technique applies when each term a_k in a summation $\sum_{k=0}^n a_k$ is independent of n .

Then for any constant $k_0 > 0$, we can write

$$\begin{aligned} \sum_{k=0}^n a_k &= \sum_{k=0}^{k_0-1} a_k + \sum_{k=k_0}^n a_k \\ &= \Theta(1) + \sum_{k=k_0}^n a_k, \end{aligned}$$

since the initial terms of the summation are all constant and there are a constant number of them. We can then use other methods to bound $\sum_{k=k_0}^n a_k$. This technique applies to infinite summations as well. For example, to find an asymptotic upper bound on

$$\sum_{k=0}^{\infty} \frac{k^2}{2^k},$$

we observe that the ratio of consecutive terms is

$$\begin{aligned} \frac{(k+1)^2/2^{k+1}}{k^2/2^k} &= \frac{(k+1)^2}{2k^2} \\ &\leq \frac{8}{9} \end{aligned}$$

if $k \geq 3$. Thus, the summation can be split into

$$\begin{aligned} \sum_{k=0}^{\infty} \frac{k^2}{2^k} &= \sum_{k=0}^2 \frac{k^2}{2^k} + \sum_{k=3}^{\infty} \frac{k^2}{2^k} \\ &\leq \sum_{k=0}^2 \frac{k^2}{2^k} + \frac{9}{8} \sum_{k=0}^{\infty} \left(\frac{8}{9}\right)^k \\ &= O(1), \end{aligned}$$

since the first summation has a constant number of terms and the second summation is a decreasing geometric series.

The technique of splitting summations can help us determine asymptotic bounds in much more difficult situations. For example, we can obtain a bound of $O(\lg n)$ on the harmonic series (A.7):

$$H_n = \sum_{k=1}^n \frac{1}{k}.$$

We do so by splitting the range 1 to n into $\lfloor \lg n \rfloor + 1$ pieces and upper-bounding the contribution of each piece by 1. For $i = 0, 1, \dots, \lfloor \lg n \rfloor$, the i th piece consists

of the terms starting at $1/2^i$ and going up to but not including $1/2^{i+1}$. The last piece might contain terms not in the original harmonic series, and thus we have

$$\begin{aligned}
 \sum_{k=1}^n \frac{1}{k} &\leq \sum_{i=0}^{\lfloor \lg n \rfloor} \sum_{j=0}^{2^i-1} \frac{1}{2^i + j} \\
 &\leq \sum_{i=0}^{\lfloor \lg n \rfloor} \sum_{j=0}^{2^i-1} \frac{1}{2^i} \\
 &= \sum_{i=0}^{\lfloor \lg n \rfloor} 1 \\
 &\leq \lg n + 1 .
 \end{aligned} \tag{A.10}$$

Approximation by integrals

When a summation has the form $\sum_{k=m}^n f(k)$, where $f(k)$ is a monotonically increasing function, we can approximate it by integrals:

$$\int_{m-1}^n f(x) dx \leq \sum_{k=m}^n f(k) \leq \int_m^{n+1} f(x) dx . \tag{A.11}$$

Figure A.1 justifies this approximation. The summation is represented as the area of the rectangles in the figure, and the integral is the shaded region under the curve. When $f(k)$ is a monotonically decreasing function, we can use a similar method to provide the bounds

$$\int_m^{n+1} f(x) dx \leq \sum_{k=m}^n f(k) \leq \int_{m-1}^n f(x) dx . \tag{A.12}$$

The integral approximation (A.12) gives a tight estimate for the n th harmonic number. For a lower bound, we obtain

$$\begin{aligned}
 \sum_{k=1}^n \frac{1}{k} &\geq \int_1^{n+1} \frac{dx}{x} \\
 &= \ln(n+1) .
 \end{aligned} \tag{A.13}$$

For the upper bound, we derive the inequality

$$\begin{aligned}
 \sum_{k=2}^n \frac{1}{k} &\leq \int_1^n \frac{dx}{x} \\
 &= \ln n ,
 \end{aligned}$$

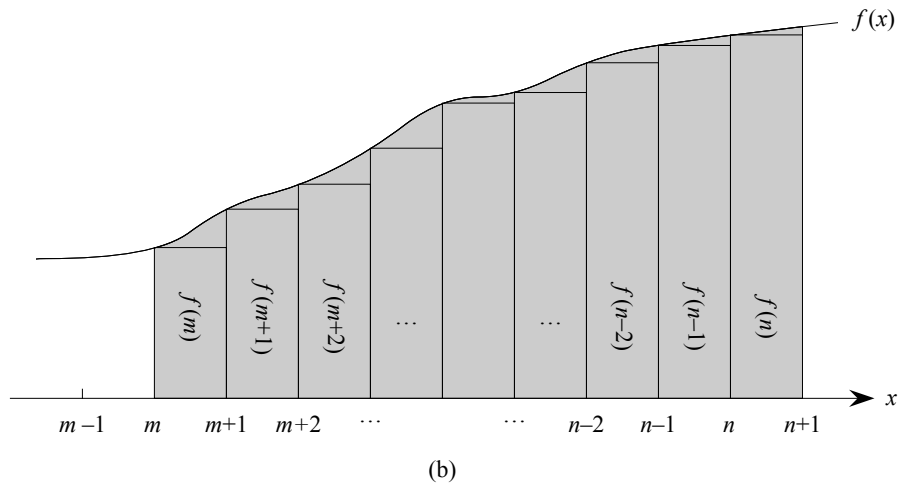
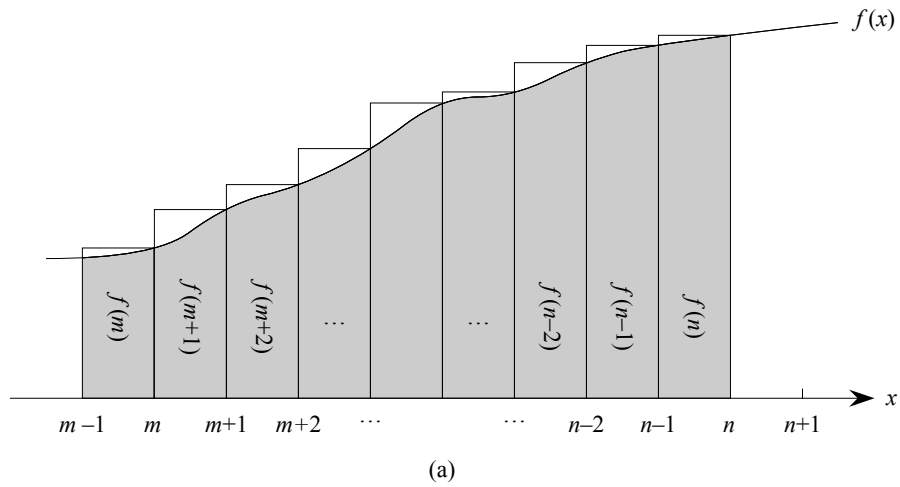


Figure A.1 Approximation of $\sum_{k=m}^n f(k)$ by integrals. The area of each rectangle is shown within the rectangle, and the total rectangle area represents the value of the summation. The integral is represented by the shaded area under the curve. By comparing areas in **(a)**, we get $\int_{m-1}^n f(x) dx \leq \sum_{k=m}^n f(k)$, and then by shifting the rectangles one unit to the right, we get $\sum_{k=m}^n f(k) \leq \int_m^{n+1} f(x) dx$ in **(b)**.

which yields the bound

$$\sum_{k=1}^n \frac{1}{k} \leq \ln n + 1. \quad (\text{A.14})$$

Exercises

A.2-1

Show that $\sum_{k=1}^n 1/k^2$ is bounded above by a constant.

A.2-2

Find an asymptotic upper bound on the summation

$$\sum_{k=0}^{\lfloor \lg n \rfloor} \lceil n/2^k \rceil.$$

A.2-3

Show that the n th harmonic number is $\Omega(\lg n)$ by splitting the summation.

A.2-4

Approximate $\sum_{k=1}^n k^3$ with an integral.

A.2-5

Why didn't we use the integral approximation (A.12) directly on $\sum_{k=1}^n 1/k$ to obtain an upper bound on the n th harmonic number?

Problems

A-1 Bounding summations

Give asymptotically tight bounds on the following summations. Assume that $r \geq 0$ and $s \geq 0$ are constants.

a. $\sum_{k=1}^n k^r.$

b. $\sum_{k=1}^n \lg^s k.$

$$c. \sum_{k=1}^n k^r \lg^s k.$$

Appendix notes

Knuth [209] provides an excellent reference for the material presented here. You can find basic properties of series in any good calculus book, such as Apostol [18] or Thomas et al. [334].

B Sets, Etc.

Many chapters of this book touch on the elements of discrete mathematics. This appendix reviews more completely the notations, definitions, and elementary properties of sets, relations, functions, graphs, and trees. If you are already well versed in this material, you can probably just skim this chapter.

B.1 Sets

A *set* is a collection of distinguishable objects, called its *members* or *elements*. If an object x is a member of a set S , we write $x \in S$ (read “ x is a member of S ” or, more briefly, “ x is in S ”). If x is not a member of S , we write $x \notin S$. We can describe a set by explicitly listing its members as a list inside braces. For example, we can define a set S to contain precisely the numbers 1, 2, and 3 by writing $S = \{1, 2, 3\}$. Since 2 is a member of the set S , we can write $2 \in S$, and since 4 is not a member, we have $4 \notin S$. A set cannot contain the same object more than once,¹ and its elements are not ordered. Two sets A and B are *equal*, written $A = B$, if they contain the same elements. For example, $\{1, 2, 3, 1\} = \{1, 2, 3\} = \{3, 2, 1\}$.

We adopt special notations for frequently encountered sets:

- $\emptyset$ denotes the *empty set*, that is, the set containing no members.
- $\mathbb{Z}$ denotes the set of *integers*, that is, the set $\{\dots, -2, -1, 0, 1, 2, \dots\}$.
- $\mathbb{R}$ denotes the set of *real numbers*.
- $\mathbb{N}$ denotes the set of *natural numbers*, that is, the set $\{0, 1, 2, \dots\}$.²

¹A variation of a set, which can contain the same object more than once, is called a *multiset*.

²Some authors start the natural numbers with 1 instead of 0. The modern trend seems to be to start with 0.

If all the elements of a set A are contained in a set B , that is, if $x \in A$ implies $x \in B$, then we write $A \subseteq B$ and say that A is a **subset** of B . A set A is a **proper subset** of B , written $A \subset B$, if $A \subseteq B$ but $A \neq B$. (Some authors use the symbol “ $\subset$ ” to denote the ordinary subset relation, rather than the proper-subset relation.) For any set A , we have $A \subseteq A$. For two sets A and B , we have $A = B$ if and only if $A \subseteq B$ and $B \subseteq A$. For any three sets A , B , and C , if $A \subseteq B$ and $B \subseteq C$, then $A \subseteq C$. For any set A , we have $\emptyset \subseteq A$.

We sometimes define sets in terms of other sets. Given a set A , we can define a set $B \subseteq A$ by stating a property that distinguishes the elements of B . For example, we can define the set of even integers by $\{x : x \in \mathbb{Z} \text{ and } x/2 \text{ is an integer}\}$. The colon in this notation is read “such that.” (Some authors use a vertical bar in place of the colon.)

Given two sets A and B , we can also define new sets by applying **set operations**:

- The **intersection** of sets A and B is the set

$$A \cap B = \{x : x \in A \text{ and } x \in B\} .$$

- The **union** of sets A and B is the set

$$A \cup B = \{x : x \in A \text{ or } x \in B\} .$$

- The **difference** between two sets A and B is the set

$$A - B = \{x : x \in A \text{ and } x \notin B\} .$$

Set operations obey the following laws:

Empty set laws:

$$A \cap \emptyset = \emptyset ,$$

$$A \cup \emptyset = A .$$

Idempotency laws:

$$A \cap A = A ,$$

$$A \cup A = A .$$

Commutative laws:

$$A \cap B = B \cap A ,$$

$$A \cup B = B \cup A .$$

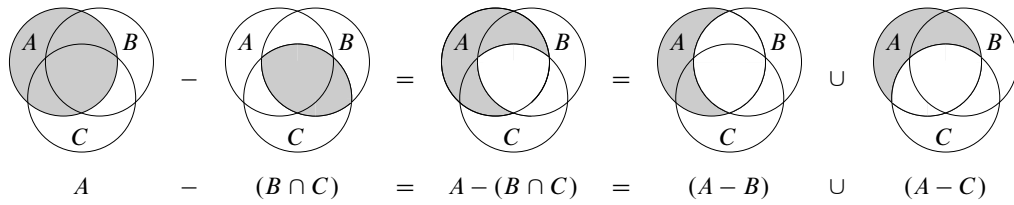


Figure B.1 A Venn diagram illustrating the first of DeMorgan's laws (B.2). Each of the sets A , B , and C is represented as a circle.

Associative laws:

$$\begin{aligned} A \cap (B \cap C) &= (A \cap B) \cap C, \\ A \cup (B \cup C) &= (A \cup B) \cup C. \end{aligned}$$

Distributive laws:

$$\begin{aligned} A \cap (B \cup C) &= (A \cap B) \cup (A \cap C), \\ A \cup (B \cap C) &= (A \cup B) \cap (A \cup C). \end{aligned} \tag{B.1}$$

Absorption laws:

$$\begin{aligned} A \cap (A \cup B) &= A, \\ A \cup (A \cap B) &= A. \end{aligned}$$

DeMorgan's laws:

$$\begin{aligned} A - (B \cap C) &= (A - B) \cup (A - C), \\ A - (B \cup C) &= (A - B) \cap (A - C). \end{aligned} \tag{B.2}$$

Figure B.1 illustrates the first of DeMorgan's laws, using a *Venn diagram*: a graphical picture in which sets are represented as regions of the plane.

Often, all the sets under consideration are subsets of some larger set U called the **universe**. For example, if we are considering various sets made up only of integers, the set $\mathbb{Z}$ of integers is an appropriate universe. Given a universe U , we define the **complement** of a set A as $\bar{A} = U - A = \{x : x \in U \text{ and } x \notin A\}$. For any set $A \subseteq U$, we have the following laws:

$$\begin{aligned} \overline{\bar{A}} &= A, \\ A \cap \bar{A} &= \emptyset, \\ A \cup \bar{A} &= U. \end{aligned}$$

We can rewrite DeMorgan's laws (B.2) with set complements. For any two sets $B, C \subseteq U$, we have

$$\begin{aligned}\overline{B \cap C} &= \overline{B} \cup \overline{C}, \\ \overline{B \cup C} &= \overline{B} \cap \overline{C}.\end{aligned}$$

Two sets A and B are **disjoint** if they have no elements in common, that is, if $A \cap B = \emptyset$. A collection $\mathcal{S} = \{S_i\}$ of nonempty sets forms a **partition** of a set S if

- the sets are **pairwise disjoint**, that is, $S_i, S_j \in \mathcal{S}$ and $i \neq j$ imply $S_i \cap S_j = \emptyset$, and
- their union is S , that is,

$$S = \bigcup_{S_i \in \mathcal{S}} S_i.$$

In other words, $\mathcal{S}$ forms a partition of S if each element of S appears in exactly one $S_i \in \mathcal{S}$.

The number of elements in a set is the **cardinality** (or **size**) of the set, denoted $|S|$. Two sets have the same cardinality if their elements can be put into a one-to-one correspondence. The cardinality of the empty set is $|\emptyset| = 0$. If the cardinality of a set is a natural number, we say the set is **finite**; otherwise, it is **infinite**. An infinite set that can be put into a one-to-one correspondence with the natural numbers $\mathbb{N}$ is **countably infinite**; otherwise, it is **uncountable**. For example, the integers $\mathbb{Z}$ are countable, but the reals $\mathbb{R}$ are uncountable.

For any two finite sets A and B , we have the identity

$$|A \cup B| = |A| + |B| - |A \cap B|, \quad (\text{B.3})$$

from which we can conclude that

$$|A \cup B| \leq |A| + |B|.$$

If A and B are disjoint, then $|A \cap B| = 0$ and thus $|A \cup B| = |A| + |B|$. If $A \subseteq B$, then $|A| \leq |B|$.

A finite set of n elements is sometimes called an ***n*-set**. A 1-set is called a **singleton**. A subset of k elements of a set is sometimes called a ***k*-subset**.

We denote the set of all subsets of a set S , including the empty set and S itself, by 2^S ; we call 2^S the **power set** of S . For example, $2^{\{a,b\}} = \{\emptyset, \{a\}, \{b\}, \{a,b\}\}$. The power set of a finite set S has cardinality $2^{|S|}$ (see Exercise B.1-5).

We sometimes care about setlike structures in which the elements are ordered. An **ordered pair** of two elements a and b is denoted (a, b) and is defined formally as the set $(a, b) = \{a, \{a, b\}\}$. Thus, the ordered pair (a, b) is *not* the same as the ordered pair (b, a) .

The **Cartesian product** of two sets A and B , denoted $A \times B$, is the set of all ordered pairs such that the first element of the pair is an element of A and the second is an element of B . More formally,

$$A \times B = \{(a, b) : a \in A \text{ and } b \in B\} .$$

For example, $\{a, b\} \times \{a, b, c\} = \{(a, a), (a, b), (a, c), (b, a), (b, b), (b, c)\}$. When A and B are finite sets, the cardinality of their Cartesian product is

$$|A \times B| = |A| \cdot |B| . \tag{B.4}$$

The Cartesian product of n sets $A_1, A_2, \dots, A_n$ is the set of **n -tuples**

$$A_1 \times A_2 \times \cdots \times A_n = \{(a_1, a_2, \dots, a_n) : a_i \in A_i \text{ for } i = 1, 2, \dots, n\} ,$$

whose cardinality is

$$|A_1 \times A_2 \times \cdots \times A_n| = |A_1| \cdot |A_2| \cdots |A_n|$$

if all sets are finite. We denote an n -fold Cartesian product over a single set A by the set

$$A^n = A \times A \times \cdots \times A ,$$

whose cardinality is $|A^n| = |A|^n$ if A is finite. We can also view an n -tuple as a finite sequence of length n (see page 1166).

Exercises

B.1-1

Draw Venn diagrams that illustrate the first of the distributive laws (B.1).

B.1-2

Prove the generalization of DeMorgan's laws to any finite collection of sets:

$$\begin{aligned} \overline{A_1 \cap A_2 \cap \cdots \cap A_n} &= \overline{A_1} \cup \overline{A_2} \cup \cdots \cup \overline{A_n} , \\ \overline{A_1 \cup A_2 \cup \cdots \cup A_n} &= \overline{A_1} \cap \overline{A_2} \cap \cdots \cap \overline{A_n} . \end{aligned}$$

B.1-3 ★

Prove the generalization of equation (B.3), which is called the *principle of inclusion and exclusion*:

$$\begin{aligned}
 |A_1 \cup A_2 \cup \cdots \cup A_n| = & \\
 & |A_1| + |A_2| + \cdots + |A_n| \\
 & - |A_1 \cap A_2| - |A_1 \cap A_3| - \cdots \quad (\text{all pairs}) \\
 & + |A_1 \cap A_2 \cap A_3| + \cdots \quad (\text{all triples}) \\
 & \vdots \\
 & + (-1)^{n-1} |A_1 \cap A_2 \cap \cdots \cap A_n|.
 \end{aligned}$$

B.1-4

Show that the set of odd natural numbers is countable.

B.1-5

Show that for any finite set S , the power set 2^S has $2^{|S|}$ elements (that is, there are $2^{|S|}$ distinct subsets of S).

B.1-6

Give an inductive definition for an n -tuple by extending the set-theoretic definition for an ordered pair.

B.2 Relations

A **binary relation** R on two sets A and B is a subset of the Cartesian product $A \times B$. If $(a, b) \in R$, we sometimes write $a R b$. When we say that R is a binary relation on a set A , we mean that R is a subset of $A \times A$. For example, the “less than” relation on the natural numbers is the set $\{(a, b) : a, b \in \mathbb{N} \text{ and } a < b\}$. An n -ary relation on sets $A_1, A_2, \dots, A_n$ is a subset of $A_1 \times A_2 \times \cdots \times A_n$.

A binary relation $R \subseteq A \times A$ is **reflexive** if

$a R a$

for all $a \in A$. For example, “=” and “ $\leq$ ” are reflexive relations on $\mathbb{N}$, but “ $<$ ” is not. The relation R is **symmetric** if

$a R b$ implies $b R a$

for all $a, b \in A$. For example, “=” is symmetric, but “ $<$ ” and “ $\leq$ ” are not. The relation R is **transitive** if

$a R b$ and $b R c$ imply $a R c$

for all $a, b, c \in A$. For example, the relations “ $<$,” “ $\leq$,” and “ $=$ ” are transitive, but the relation $R = \{(a, b) : a, b \in \mathbb{N} \text{ and } a = b - 1\}$ is not, since $3 R 4$ and $4 R 5$ do not imply $3 R 5$.

A relation that is reflexive, symmetric, and transitive is an **equivalence relation**. For example, “ $=$ ” is an equivalence relation on the natural numbers, but “ $<$ ” is not. If R is an equivalence relation on a set A , then for $a \in A$, the **equivalence class** of a is the set $[a] = \{b \in A : a R b\}$, that is, the set of all elements equivalent to a . For example, if we define $R = \{(a, b) : a, b \in \mathbb{N} \text{ and } a + b \text{ is an even number}\}$, then R is an equivalence relation, since $a + a$ is even (reflexive), $a + b$ is even implies $b + a$ is even (symmetric), and $a + b$ is even and $b + c$ is even imply $a + c$ is even (transitive). The equivalence class of 4 is $[4] = \{0, 2, 4, 6, \dots\}$, and the equivalence class of 3 is $[3] = \{1, 3, 5, 7, \dots\}$. A basic theorem of equivalence classes is the following.

Theorem B.1 (An equivalence relation is the same as a partition)

The equivalence classes of any equivalence relation R on a set A form a partition of A , and any partition of A determines an equivalence relation on A for which the sets in the partition are the equivalence classes.

Proof For the first part of the proof, we must show that the equivalence classes of R are nonempty, pairwise-disjoint sets whose union is A . Because R is reflexive, $a \in [a]$, and so the equivalence classes are nonempty; moreover, since every element $a \in A$ belongs to the equivalence class $[a]$, the union of the equivalence classes is A . It remains to show that the equivalence classes are pairwise disjoint, that is, if two equivalence classes $[a]$ and $[b]$ have an element c in common, then they are in fact the same set. Suppose that $a R c$ and $b R c$. By symmetry, $c R b$, and by transitivity, $a R b$. Thus, for any arbitrary element $x \in [a]$, we have $x R a$ and, by transitivity, $x R b$, and thus $[a] \subseteq [b]$. Similarly, $[b] \subseteq [a]$, and thus $[a] = [b]$.

For the second part of the proof, let $\mathcal{A} = \{A_i\}$ be a partition of A , and define $R = \{(a, b) : \text{there exists } i \text{ such that } a \in A_i \text{ and } b \in A_i\}$. We claim that R is an equivalence relation on A . Reflexivity holds, since $a \in A_i$ implies $a R a$. Symmetry holds, because if $a R b$, then a and b are in the same set A_i , and hence $b R a$. If $a R b$ and $b R c$, then all three elements are in the same set A_i , and thus $a R c$ and transitivity holds. To see that the sets in the partition are the equivalence classes of R , observe that if $a \in A_i$, then $x \in [a]$ implies $x \in A_i$, and $x \in A_i$ implies $x \in [a]$. ■

A binary relation R on a set A is **antisymmetric** if
 $a R b$ and $b R a$ imply $a = b$.

For example, the “ $\leq$ ” relation on the natural numbers is antisymmetric, since $a \leq b$ and $b \leq a$ imply $a = b$. A relation that is reflexive, antisymmetric, and transitive is a **partial order**, and we call a set on which a partial order is defined a **partially ordered set**. For example, the relation “is a descendant of” is a partial order on the set of all people (if we view individuals as being their own descendants).

In a partially ordered set A , there may be no single “maximum” element a such that $b R a$ for all $b \in A$. Instead, the set may contain several **maximal** elements a such that for no $b \in A$, where $b \neq a$, is it the case that $a R b$. For example, a collection of different-sized boxes may contain several maximal boxes that don’t fit inside any other box, yet it has no single “maximum” box into which any other box will fit.³

A relation R on a set A is a **total relation** if for all $a, b \in A$, we have $a R b$ or $b R a$ (or both), that is, if every pairing of elements of A is related by R . A partial order that is also a total relation is a **total order** or **linear order**. For example, the relation “ $\leq$ ” is a total order on the natural numbers, but the “is a descendant of” relation is not a total order on the set of all people, since there are individuals neither of whom is descended from the other. A total relation that is transitive, but not necessarily reflexive and antisymmetric, is a **total preorder**.

Exercises

B.2-1

Prove that the subset relation “ $\subseteq$ ” on all subsets of $\mathbb{Z}$ is a partial order but not a total order.

B.2-2

Show that for any positive integer n , the relation “equivalent modulo n ” is an equivalence relation on the integers. (We say that $a \equiv b \pmod{n}$ if there exists an integer q such that $a - b = qn$.) Into what equivalence classes does this relation partition the integers?

B.2-3

Give examples of relations that are

- a. reflexive and symmetric but not transitive,
- b. reflexive and transitive but not symmetric,
- c. symmetric and transitive but not reflexive.

³To be precise, in order for the “fit inside” relation to be a partial order, we need to view a box as fitting inside itself.

B.2-4

Let S be a finite set, and let R be an equivalence relation on $S \times S$. Show that if in addition R is antisymmetric, then the equivalence classes of S with respect to R are singletons.

B.2-5

Professor Narcissus claims that if a relation R is symmetric and transitive, then it is also reflexive. He offers the following proof. By symmetry, $a R b$ implies $b R a$. Transitivity, therefore, implies $a R a$. Is the professor correct?

B.3 Functions

Given two sets A and B , a **function** f is a binary relation on A and B such that for all $a \in A$, there exists precisely one $b \in B$ such that $(a, b) \in f$. The set A is called the **domain** of f , and the set B is called the **codomain** of f . We sometimes write $f : A \rightarrow B$; and if $(a, b) \in f$, we write $b = f(a)$, since b is uniquely determined by the choice of a .

Intuitively, the function f assigns an element of B to each element of A . No element of A is assigned two different elements of B , but the same element of B can be assigned to two different elements of A . For example, the binary relation

$$f = \{(a, b) : a, b \in \mathbb{N} \text{ and } b = a \bmod 2\}$$

is a function $f : \mathbb{N} \rightarrow \{0, 1\}$, since for each natural number a , there is exactly one value b in $\{0, 1\}$ such that $b = a \bmod 2$. For this example, $0 = f(0)$, $1 = f(1)$, $0 = f(2)$, etc. In contrast, the binary relation

$$g = \{(a, b) : a, b \in \mathbb{N} \text{ and } a + b \text{ is even}\}$$

is not a function, since $(1, 3)$ and $(1, 5)$ are both in g , and thus for the choice $a = 1$, there is not precisely one b such that $(a, b) \in g$.

Given a function $f : A \rightarrow B$, if $b = f(a)$, we say that a is the **argument** of f and that b is the **value** of f at a . We can define a function by stating its value for every element of its domain. For example, we might define $f(n) = 2n$ for $n \in \mathbb{N}$, which means $f = \{(n, 2n) : n \in \mathbb{N}\}$. Two functions f and g are **equal** if they have the same domain and codomain and if, for all a in the domain, $f(a) = g(a)$.

A **finite sequence** of length n is a function f whose domain is the set of n integers $\{0, 1, \dots, n-1\}$. We often denote a finite sequence by listing its values: $\langle f(0), f(1), \dots, f(n-1) \rangle$. An **infinite sequence** is a function whose domain is the set $\mathbb{N}$ of natural numbers. For example, the Fibonacci sequence, defined by recurrence (3.22), is the infinite sequence $\langle 0, 1, 1, 2, 3, 5, 8, 13, 21, \dots \rangle$.

When the domain of a function f is a Cartesian product, we often omit the extra parentheses surrounding the argument of f . For example, if we had a function $f : A_1 \times A_2 \times \cdots \times A_n \rightarrow B$, we would write $b = f(a_1, a_2, \dots, a_n)$ instead of $b = f((a_1, a_2, \dots, a_n))$. We also call each a_i an **argument** to the function f , though technically the (single) argument to f is the n -tuple $(a_1, a_2, \dots, a_n)$.

If $f : A \rightarrow B$ is a function and $b = f(a)$, then we sometimes say that b is the **image** of a under f . The image of a set $A' \subseteq A$ under f is defined by

$$f(A') = \{b \in B : b = f(a) \text{ for some } a \in A'\}.$$

The **range** of f is the image of its domain, that is, $f(A)$. For example, the range of the function $f : \mathbb{N} \rightarrow \mathbb{N}$ defined by $f(n) = 2n$ is $f(\mathbb{N}) = \{m : m = 2n \text{ for some } n \in \mathbb{N}\}$, in other words, the set of nonnegative even integers.

A function is a **surjection** if its range is its codomain. For example, the function $f(n) = \lfloor n/2 \rfloor$ is a surjective function from $\mathbb{N}$ to $\mathbb{N}$, since every element in $\mathbb{N}$ appears as the value of f for some argument. In contrast, the function $f(n) = 2n$ is not a surjective function from $\mathbb{N}$ to $\mathbb{N}$, since no argument to f can produce 3 as a value. The function $f(n) = 2n$ is, however, a surjective function from the natural numbers to the even numbers. A surjection $f : A \rightarrow B$ is sometimes described as mapping A **onto** B . When we say that f is onto, we mean that it is surjective.

A function $f : A \rightarrow B$ is an **injection** if distinct arguments to f produce distinct values, that is, if $a \neq a'$ implies $f(a) \neq f(a')$. For example, the function $f(n) = 2n$ is an injective function from $\mathbb{N}$ to $\mathbb{N}$, since each even number b is the image under f of at most one element of the domain, namely $b/2$. The function $f(n) = \lfloor n/2 \rfloor$ is not injective, since the value 1 is produced by two arguments: 2 and 3. An injection is sometimes called a **one-to-one** function.

A function $f : A \rightarrow B$ is a **bijection** if it is injective and surjective. For example, the function $f(n) = (-1)^n \lfloor n/2 \rfloor$ is a bijection from $\mathbb{N}$ to $\mathbb{Z}$:

$$\begin{array}{ll} 0 & \rightarrow 0, \\ 1 & \rightarrow -1, \\ 2 & \rightarrow 1, \\ 3 & \rightarrow -2, \\ 4 & \rightarrow 2, \\ & \vdots \end{array}$$

The function is injective, since no element of $\mathbb{Z}$ is the image of more than one element of $\mathbb{N}$. It is surjective, since every element of $\mathbb{Z}$ appears as the image of some element of $\mathbb{N}$. Hence, the function is bijective. A bijection is sometimes called a **one-to-one correspondence**, since it pairs elements in the domain and codomain. A bijection from a set A to itself is sometimes called a **permutation**.

When a function f is bijective, we define its **inverse** f^{-1} as $f^{-1}(b) = a$ if and only if $f(a) = b$.

For example, the inverse of the function $f(n) = (-1)^n \lceil n/2 \rceil$ is

$$f^{-1}(m) = \begin{cases} 2m & \text{if } m \geq 0, \\ -2m - 1 & \text{if } m < 0. \end{cases}$$

Exercises

B.3-1

Let A and B be finite sets, and let $f : A \rightarrow B$ be a function. Show that

- a. if f is injective, then $|A| \leq |B|$;
- b. if f is surjective, then $|A| \geq |B|$.

B.3-2

Is the function $f(x) = x + 1$ bijective when the domain and the codomain are $\mathbb{N}$? Is it bijective when the domain and the codomain are $\mathbb{Z}$?

B.3-3

Give a natural definition for the inverse of a binary relation such that if a relation is in fact a bijective function, its relational inverse is its functional inverse.

B.3-4 ★

Give a bijection from $\mathbb{Z}$ to $\mathbb{Z} \times \mathbb{Z}$.

B.4 Graphs

This section presents two kinds of graphs: directed and undirected. Certain definitions in the literature differ from those given here, but for the most part, the differences are slight. Section 22.1 shows how we can represent graphs in computer memory.

A **directed graph** (or **digraph**) G is a pair (V, E) , where V is a finite set and E is a binary relation on V . The set V is called the **vertex set** of G , and its elements are called **vertices** (singular: **vertex**). The set E is called the **edge set** of G , and its elements are called **edges**. Figure B.2(a) is a pictorial representation of a directed graph on the vertex set $\{1, 2, 3, 4, 5, 6\}$. Vertices are represented by circles in the figure, and edges are represented by arrows. Note that **self-loops**—edges from a vertex to itself—are possible.

In an **undirected graph** $G = (V, E)$, the edge set E consists of *unordered* pairs of vertices, rather than ordered pairs. That is, an edge is a set $\{u, v\}$, where

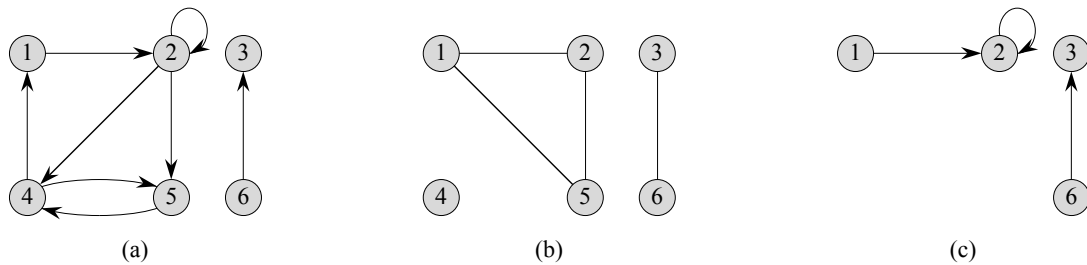


Figure B.2 Directed and undirected graphs. **(a)** A directed graph $G = (V, E)$, where $V = \{1, 2, 3, 4, 5, 6\}$ and $E = \{(1, 2), (2, 2), (2, 4), (2, 5), (4, 1), (4, 5), (5, 4), (5, 6), (6, 3)\}$. The edge $(2, 2)$ is a self-loop. **(b)** An undirected graph $G = (V, E)$, where $V = \{1, 2, 3, 4, 5, 6\}$ and $E = \{(1, 2), (1, 5), (2, 5), (3, 6)\}$. The vertex 4 is isolated. **(c)** The subgraph of the graph in part (a) induced by the vertex set $\{1, 2, 3, 6\}$.

$u, v \in V$ and $u \neq v$. By convention, we use the notation (u, v) for an edge, rather than the set notation $\{u, v\}$, and we consider (u, v) and (v, u) to be the same edge. In an undirected graph, self-loops are forbidden, and so every edge consists of two distinct vertices. Figure B.2(b) is a pictorial representation of an undirected graph on the vertex set $\{1, 2, 3, 4, 5, 6\}$.

Many definitions for directed and undirected graphs are the same, although certain terms have slightly different meanings in the two contexts. If (u, v) is an edge in a directed graph $G = (V, E)$, we say that (u, v) is **incident from** or **leaves** vertex u and is **incident to** or **enters** vertex v . For example, the edges leaving vertex 2 in Figure B.2(a) are $(2, 2)$, $(2, 4)$, and $(2, 5)$. The edges entering vertex 2 are $(1, 2)$ and $(2, 2)$. If (u, v) is an edge in an undirected graph $G = (V, E)$, we say that (u, v) is **incident on** vertices u and v . In Figure B.2(b), the edges incident on vertex 2 are $(1, 2)$ and $(2, 5)$.

If (u, v) is an edge in a graph $G = (V, E)$, we say that vertex v is **adjacent** to vertex u . When the graph is undirected, the adjacency relation is symmetric. When the graph is directed, the adjacency relation is not necessarily symmetric. If v is adjacent to u in a directed graph, we sometimes write $u \rightarrow v$. In parts (a) and (b) of Figure B.2, vertex 2 is adjacent to vertex 1, since the edge $(1, 2)$ belongs to both graphs. Vertex 1 is *not* adjacent to vertex 2 in Figure B.2(a), since the edge $(2, 1)$ does not belong to the graph.

The **degree** of a vertex in an undirected graph is the number of edges incident on it. For example, vertex 2 in Figure B.2(b) has degree 2. A vertex whose degree is 0, such as vertex 4 in Figure B.2(b), is **isolated**. In a directed graph, the **out-degree** of a vertex is the number of edges leaving it, and the **in-degree** of a vertex is the number of edges entering it. The **degree** of a vertex in a directed graph is its in-

degree plus its out-degree. Vertex 2 in Figure B.2(a) has in-degree 2, out-degree 3, and degree 5.

A **path** of **length** k from a vertex u to a vertex u' in a graph $G = (V, E)$ is a sequence $\langle v_0, v_1, v_2, \dots, v_k \rangle$ of vertices such that $u = v_0$, $u' = v_k$, and $(v_{i-1}, v_i) \in E$ for $i = 1, 2, \dots, k$. The length of the path is the number of edges in the path. The path **contains** the vertices $v_0, v_1, \dots, v_k$ and the edges $(v_0, v_1), (v_1, v_2), \dots, (v_{k-1}, v_k)$. (There is always a 0-length path from u to u .) If there is a path p from u to u' , we say that u' is **reachable** from u via p , which we sometimes write as $u \xrightarrow{p} u'$ if G is directed. A path is **simple**⁴ if all vertices in the path are distinct. In Figure B.2(a), the path $\langle 1, 2, 5, 4 \rangle$ is a simple path of length 3. The path $\langle 2, 5, 4, 5 \rangle$ is not simple.

A **subpath** of path $p = \langle v_0, v_1, \dots, v_k \rangle$ is a contiguous subsequence of its vertices. That is, for any $0 \leq i \leq j \leq k$, the subsequence of vertices $\langle v_i, v_{i+1}, \dots, v_j \rangle$ is a subpath of p .

In a directed graph, a path $\langle v_0, v_1, \dots, v_k \rangle$ forms a **cycle** if $v_0 = v_k$ and the path contains at least one edge. The cycle is **simple** if, in addition, $v_1, v_2, \dots, v_k$ are distinct. A self-loop is a cycle of length 1. Two paths $\langle v_0, v_1, v_2, \dots, v_{k-1}, v_0 \rangle$ and $\langle v'_0, v'_1, v'_2, \dots, v'_{k-1}, v'_0 \rangle$ form the same cycle if there exists an integer j such that $v'_i = v_{(i+j) \bmod k}$ for $i = 0, 1, \dots, k-1$. In Figure B.2(a), the path $\langle 1, 2, 4, 1 \rangle$ forms the same cycle as the paths $\langle 2, 4, 1, 2 \rangle$ and $\langle 4, 1, 2, 4 \rangle$. This cycle is simple, but the cycle $\langle 1, 2, 4, 5, 4, 1 \rangle$ is not. The cycle $\langle 2, 2 \rangle$ formed by the edge $(2, 2)$ is a self-loop. A directed graph with no self-loops is **simple**. In an undirected graph, a path $\langle v_0, v_1, \dots, v_k \rangle$ forms a **cycle** if $k > 0$, $v_0 = v_k$, and all edges on the path are distinct; the cycle is **simple** if $v_1, v_2, \dots, v_k$ are distinct. For example, in Figure B.2(b), the path $\langle 1, 2, 5, 1 \rangle$ is a simple cycle. A graph with no simple cycles is **acyclic**.

An undirected graph is **connected** if every vertex is reachable from all other vertices. The **connected components** of an undirected graph are the equivalence classes of vertices under the “is reachable from” relation. The graph in Figure B.2(b) has three connected components: $\{1, 2, 5\}$, $\{3, 6\}$, and $\{4\}$. Every vertex in $\{1, 2, 5\}$ is reachable from every other vertex in $\{1, 2, 5\}$. An undirected graph is connected if it has exactly one connected component. The edges of a connected component are those that are incident on only the vertices of the component; in other words, edge (u, v) is an edge of a connected component only if both u and v are vertices of the component.

⁴Some authors refer to what we call a path as a “walk” and to what we call a simple path as just a “path.” We use the terms “path” and “simple path” throughout this book in a manner consistent with their definitions.

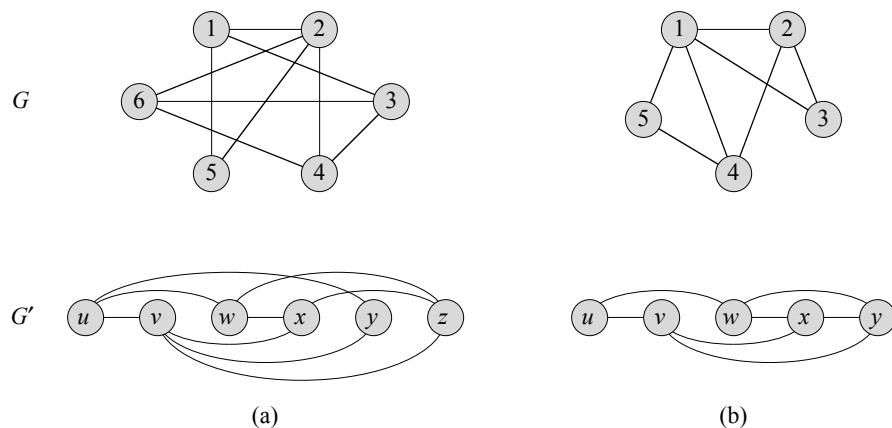


Figure B.3 (a) A pair of isomorphic graphs. The vertices of the top graph are mapped to the vertices of the bottom graph by $f(1) = u, f(2) = v, f(3) = w, f(4) = x, f(5) = y, f(6) = z$. (b) Two graphs that are not isomorphic, since the top graph has a vertex of degree 4 and the bottom graph does not.

A directed graph is **strongly connected** if every two vertices are reachable from each other. The **strongly connected components** of a directed graph are the equivalence classes of vertices under the “are mutually reachable” relation. A directed graph is strongly connected if it has only one strongly connected component. The graph in Figure B.2(a) has three strongly connected components: $\{1, 2, 4, 5\}$, $\{3\}$, and $\{6\}$. All pairs of vertices in $\{1, 2, 4, 5\}$ are mutually reachable. The vertices $\{3, 6\}$ do not form a strongly connected component, since vertex 6 cannot be reached from vertex 3.

Two graphs $G = (V, E)$ and $G' = (V', E')$ are **isomorphic** if there exists a bijection $f : V \rightarrow V'$ such that $(u, v) \in E$ if and only if $(f(u), f(v)) \in E'$. In other words, we can relabel the vertices of G to be vertices of G' , maintaining the corresponding edges in G and G' . Figure B.3(a) shows a pair of isomorphic graphs G and G' with respective vertex sets $V = \{1, 2, 3, 4, 5, 6\}$ and $V' = \{u, v, w, x, y, z\}$. The mapping from V to V' given by $f(1) = u, f(2) = v, f(3) = w, f(4) = x, f(5) = y, f(6) = z$ provides the required bijective function. The graphs in Figure B.3(b) are not isomorphic. Although both graphs have 5 vertices and 7 edges, the top graph has a vertex of degree 4 and the bottom graph does not.

We say that a graph $G' = (V', E')$ is a **subgraph** of $G = (V, E)$ if $V' \subseteq V$ and $E' \subseteq E$. Given a set $V' \subseteq V$, the subgraph of G **induced** by V' is the graph $G' = (V', E')$, where

$$E' = \{(u, v) \in E : u, v \in V'\} .$$

The subgraph induced by the vertex set $\{1, 2, 3, 6\}$ in Figure B.2(a) appears in Figure B.2(c) and has the edge set $\{(1, 2), (2, 2), (6, 3)\}$.

Given an undirected graph $G = (V, E)$, the **directed version** of G is the directed graph $G' = (V, E')$, where $(u, v) \in E'$ if and only if $(u, v) \in E$. That is, we replace each undirected edge (u, v) in G by the two directed edges (u, v) and (v, u) in the directed version. Given a directed graph $G = (V, E)$, the **undirected version** of G is the undirected graph $G' = (V, E')$, where $(u, v) \in E'$ if and only if $u \neq v$ and E contains at least one of the edges (u, v) and (v, u) . That is, the undirected version contains the edges of G “with their directions removed” and with self-loops eliminated. (Since (u, v) and (v, u) are the same edge in an undirected graph, the undirected version of a directed graph contains it only once, even if the directed graph contains both edges (u, v) and (v, u) .) In a directed graph $G = (V, E)$, a **neighbor** of a vertex u is any vertex that is adjacent to u in the undirected version of G . That is, v is a neighbor of u if $u \neq v$ and either $(u, v) \in E$ or $(v, u) \in E$. In an undirected graph, u and v are neighbors if they are adjacent.

Several kinds of graphs have special names. A **complete graph** is an undirected graph in which every pair of vertices is adjacent. A **bipartite graph** is an undirected graph $G = (V, E)$ in which V can be partitioned into two sets V_1 and V_2 such that $(u, v) \in E$ implies either $u \in V_1$ and $v \in V_2$ or $u \in V_2$ and $v \in V_1$. That is, all edges go between the two sets V_1 and V_2 . An acyclic, undirected graph is a **forest**, and a connected, acyclic, undirected graph is a (**free**) **tree** (see Section B.5). We often take the first letters of “directed acyclic graph” and call such a graph a **dag**.

There are two variants of graphs that you may occasionally encounter. A **multi-graph** is like an undirected graph, but it can have both multiple edges between vertices and self-loops. A **hypergraph** is like an undirected graph, but each **hyperedge**, rather than connecting two vertices, connects an arbitrary subset of vertices. Many algorithms written for ordinary directed and undirected graphs can be adapted to run on these graphlike structures.

The **contraction** of an undirected graph $G = (V, E)$ by an edge $e = (u, v)$ is a graph $G' = (V', E')$, where $V' = V - \{u, v\} \cup \{x\}$ and x is a new vertex. The set of edges E' is formed from E by deleting the edge (u, v) and, for each vertex w adjacent to u or v , deleting whichever of (u, w) and (v, w) is in E and adding the new edge (x, w) . In effect, u and v are “contracted” into a single vertex.

Exercises

B.4-1

Attendees of a faculty party shake hands to greet each other, and each professor remembers how many times he or she shook hands. At the end of the party, the department head adds up the number of times that each professor shook hands.

Show that the result is even by proving the **handshaking lemma**: if $G = (V, E)$ is an undirected graph, then

$$\sum_{v \in V} \text{degree}(v) = 2|E|.$$

B.4-2

Show that if a directed or undirected graph contains a path between two vertices u and v , then it contains a simple path between u and v . Show that if a directed graph contains a cycle, then it contains a simple cycle.

B.4-3

Show that any connected, undirected graph $G = (V, E)$ satisfies $|E| \geq |V| - 1$.

B.4-4

Verify that in an undirected graph, the “is reachable from” relation is an equivalence relation on the vertices of the graph. Which of the three properties of an equivalence relation hold in general for the “is reachable from” relation on the vertices of a directed graph?

B.4-5

What is the undirected version of the directed graph in Figure B.2(a)? What is the directed version of the undirected graph in Figure B.2(b)?

B.4-6 ★

Show that we can represent a hypergraph by a bipartite graph if we let incidence in the hypergraph correspond to adjacency in the bipartite graph. (*Hint*: Let one set of vertices in the bipartite graph correspond to vertices of the hypergraph, and let the other set of vertices of the bipartite graph correspond to hyperedges.)

B.5 Trees

As with graphs, there are many related, but slightly different, notions of trees. This section presents definitions and mathematical properties of several kinds of trees. Sections 10.4 and 22.1 describe how we can represent trees in computer memory.

B.5.1 Free trees

As defined in Section B.4, a **free tree** is a connected, acyclic, undirected graph. We often omit the adjective “free” when we say that a graph is a tree. If an undirected graph is acyclic but possibly disconnected, it is a **forest**. Many algorithms that work

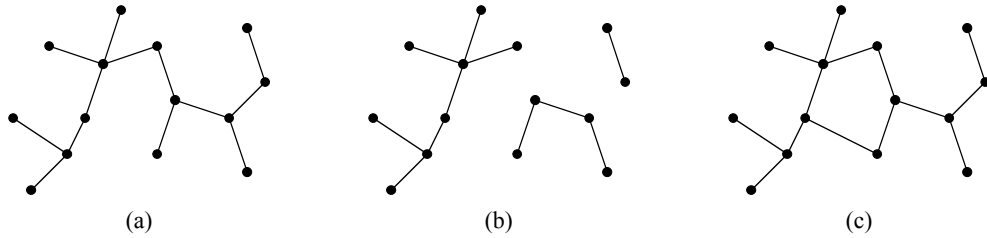


Figure B.4 (a) A free tree. (b) A forest. (c) A graph that contains a cycle and is therefore neither a tree nor a forest.

for trees also work for forests. Figure B.4(a) shows a free tree, and Figure B.4(b) shows a forest. The forest in Figure B.4(b) is not a tree because it is not connected. The graph in Figure B.4(c) is connected but neither a tree nor a forest, because it contains a cycle.

The following theorem captures many important facts about free trees.

Theorem B.2 (Properties of free trees)

Let $G = (V, E)$ be an undirected graph. The following statements are equivalent.

1. G is a free tree.
2. Any two vertices in G are connected by a unique simple path.
3. G is connected, but if any edge is removed from E , the resulting graph is disconnected.
4. G is connected, and $|E| = |V| - 1$.
5. G is acyclic, and $|E| = |V| - 1$.
6. G is acyclic, but if any edge is added to E , the resulting graph contains a cycle.

Proof (1) $\Rightarrow$ (2): Since a tree is connected, any two vertices in G are connected by at least one simple path. Suppose, for the sake of contradiction, that vertices u and v are connected by two distinct simple paths p_1 and p_2 , as shown in Figure B.5. Let w be the vertex at which the paths first diverge; that is, w is the first vertex on both p_1 and p_2 whose successor on p_1 is x and whose successor on p_2 is y , where $x \neq y$. Let z be the first vertex at which the paths reconverge; that is, z is the first vertex following w on p_1 that is also on p_2 . Let p' be the subpath of p_1 from w through x to z , and let p'' be the subpath of p_2 from w through y to z . Paths p' and p'' share no vertices except their endpoints. Thus, the path obtained by concatenating p' and the reverse of p'' is a cycle, which contradicts our assumption

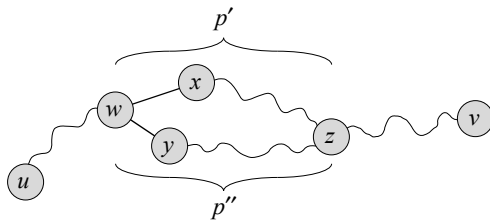


Figure B.5 A step in the proof of Theorem B.2: if (1) G is a free tree, then (2) any two vertices in G are connected by a unique simple path. Assume for the sake of contradiction that vertices u and v are connected by two distinct simple paths p_1 and p_2 . These paths first diverge at vertex w , and they first reconverge at vertex z . The path p' concatenated with the reverse of the path p'' forms a cycle, which yields the contradiction.

that G is a tree. Thus, if G is a tree, there can be at most one simple path between two vertices.

(2) $\Rightarrow$ (3): If any two vertices in G are connected by a unique simple path, then G is connected. Let (u, v) be any edge in E . This edge is a path from u to v , and so it must be the unique path from u to v . If we remove (u, v) from G , there is no path from u to v , and hence its removal disconnects G .

(3) $\Rightarrow$ (4): By assumption, the graph G is connected, and by Exercise B.4-3, we have $|E| \geq |V| - 1$. We shall prove $|E| \leq |V| - 1$ by induction. A connected graph with $n = 1$ or $n = 2$ vertices has $n - 1$ edges. Suppose that G has $n \geq 3$ vertices and that all graphs satisfying (3) with fewer than n vertices also satisfy $|E| \leq |V| - 1$. Removing an arbitrary edge from G separates the graph into $k \geq 2$ connected components (actually $k = 2$). Each component satisfies (3), or else G would not satisfy (3). If we view each connected component V_i , with edge set E_i , as its own free tree, then because each component has fewer than $|V|$ vertices, by the inductive hypothesis we have $|E_i| \leq |V_i| - 1$. Thus, the number of edges in all components combined is at most $|V| - k \leq |V| - 2$. Adding in the removed edge yields $|E| \leq |V| - 1$.

(4) $\Rightarrow$ (5): Suppose that G is connected and that $|E| = |V| - 1$. We must show that G is acyclic. Suppose that G has a cycle containing k vertices $v_1, v_2, \dots, v_k$, and without loss of generality assume that this cycle is simple. Let $G_k = (V_k, E_k)$ be the subgraph of G consisting of the cycle. Note that $|V_k| = |E_k| = k$. If $k < |V|$, there must be a vertex $v_{k+1} \in V - V_k$ that is adjacent to some vertex $v_i \in V_k$, since G is connected. Define $G_{k+1} = (V_{k+1}, E_{k+1})$ to be the subgraph of G with $V_{k+1} = V_k \cup \{v_{k+1}\}$ and $E_{k+1} = E_k \cup \{(v_i, v_{k+1})\}$. Note that $|V_{k+1}| = |E_{k+1}| = k + 1$. If $k + 1 < |V|$, we can continue, defining G_{k+2} in the same manner, and so forth, until we obtain $G_n = (V_n, E_n)$, where $n = |V|$,

$V_n = V$, and $|E_n| = |V_n| = |V|$. Since G_n is a subgraph of G , we have $E_n \subseteq E$, and hence $|E| \geq |V|$, which contradicts the assumption that $|E| = |V| - 1$. Thus, G is acyclic.

(5) $\Rightarrow$ (6): Suppose that G is acyclic and that $|E| = |V| - 1$. Let k be the number of connected components of G . Each connected component is a free tree by definition, and since (1) implies (5), the sum of all edges in all connected components of G is $|V| - k$. Consequently, we must have $k = 1$, and G is in fact a tree. Since (1) implies (2), any two vertices in G are connected by a unique simple path. Thus, adding any edge to G creates a cycle.

(6) $\Rightarrow$ (1): Suppose that G is acyclic but that adding any edge to E creates a cycle. We must show that G is connected. Let u and v be arbitrary vertices in G . If u and v are not already adjacent, adding the edge (u, v) creates a cycle in which all edges but (u, v) belong to G . Thus, the cycle minus edge (u, v) must contain a path from u to v , and since u and v were chosen arbitrarily, G is connected. ■

B.5.2 Rooted and ordered trees

A **rooted tree** is a free tree in which one of the vertices is distinguished from the others. We call the distinguished vertex the **root** of the tree. We often refer to a vertex of a rooted tree as a **node**⁵ of the tree. Figure B.6(a) shows a rooted tree on a set of 12 nodes with root 7.

Consider a node x in a rooted tree T with root r . We call any node y on the unique simple path from r to x an **ancestor** of x . If y is an ancestor of x , then x is a **descendant** of y . (Every node is both an ancestor and a descendant of itself.) If y is an ancestor of x and $x \neq y$, then y is a **proper ancestor** of x and x is a **proper descendant** of y . The **subtree rooted at x** is the tree induced by descendants of x , rooted at x . For example, the subtree rooted at node 8 in Figure B.6(a) contains nodes 8, 6, 5, and 9.

If the last edge on the simple path from the root r of a tree T to a node x is (y, x) , then y is the **parent** of x , and x is a **child** of y . The root is the only node in T with no parent. If two nodes have the same parent, they are **siblings**. A node with no children is a **leaf** or **external node**. A nonleaf node is an **internal node**.

⁵The term “node” is often used in the graph theory literature as a synonym for “vertex.” We reserve the term “node” to mean a vertex of a rooted tree.

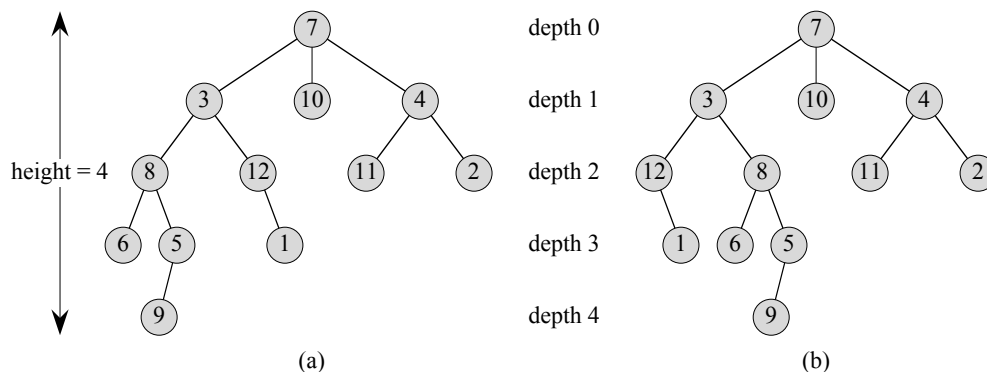


Figure B.6 Rooted and ordered trees. **(a)** A rooted tree with height 4. The tree is drawn in a standard way: the root (node 7) is at the top, its children (nodes with depth 1) are beneath it, their children (nodes with depth 2) are beneath them, and so forth. If the tree is ordered, the relative left-to-right order of the children of a node matters; otherwise it doesn't. **(b)** Another rooted tree. As a rooted tree, it is identical to the tree in (a), but as an ordered tree it is different, since the children of node 3 appear in a different order.

The number of children of a node x in a rooted tree T equals the *degree* of x .⁶ The length of the simple path from the root r to a node x is the *depth* of x in T . A *level* of a tree consists of all nodes at the same depth. The *height* of a node in a tree is the number of edges on the longest simple downward path from the node to a leaf, and the height of a tree is the height of its root. The height of a tree is also equal to the largest depth of any node in the tree.

An *ordered tree* is a rooted tree in which the children of each node are ordered. That is, if a node has k children, then there is a first child, a second child, ..., and a k th child. The two trees in Figure B.6 are different when considered to be ordered trees, but the same when considered to be just rooted trees.

B.5.3 Binary and positional trees

We define binary trees recursively. A *binary tree* T is a structure defined on a finite set of nodes that either

- contains no nodes, or

⁶Notice that the degree of a node depends on whether we consider T to be a rooted tree or a free tree. The degree of a vertex in a free tree is, as in any undirected graph, the number of adjacent vertices. In a rooted tree, however, the degree is the number of children—the parent of a node does not count toward its degree.

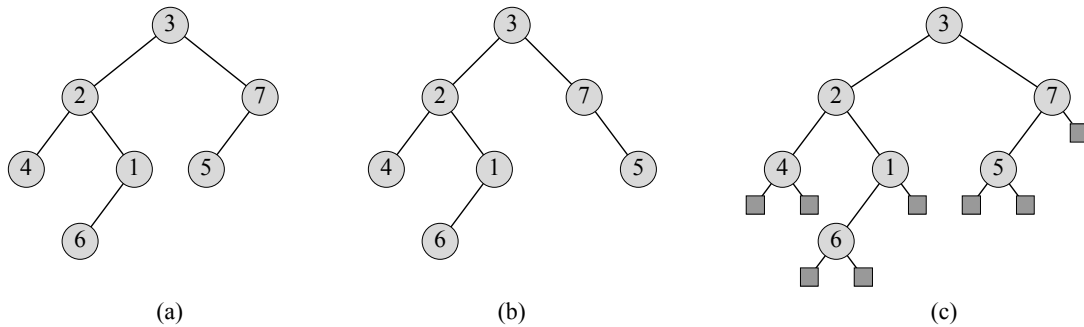


Figure B.7 Binary trees. **(a)** A binary tree drawn in a standard way. The left child of a node is drawn beneath the node and to the left. The right child is drawn beneath and to the right. **(b)** A binary tree different from the one in (a). In (a), the left child of node 7 is 5 and the right child is absent. In (b), the left child of node 7 is absent and the right child is 5. As ordered trees, these trees are the same, but as binary trees, they are distinct. **(c)** The binary tree in (a) represented by the internal nodes of a full binary tree: an ordered tree in which each internal node has degree 2. The leaves in the tree are shown as squares.

- is composed of three disjoint sets of nodes: a **root** node, a binary tree called its **left subtree**, and a binary tree called its **right subtree**.

The binary tree that contains no nodes is called the **empty tree** or **null tree**, sometimes denoted NIL. If the left subtree is nonempty, its root is called the **left child** of the root of the entire tree. Likewise, the root of a nonnull right subtree is the **right child** of the root of the entire tree. If a subtree is the null tree NIL, we say that the child is **absent** or **missing**. Figure B.7(a) shows a binary tree.

A binary tree is not simply an ordered tree in which each node has degree at most 2. For example, in a binary tree, if a node has just one child, the position of the child—whether it is the **left child** or the **right child**—matters. In an ordered tree, there is no distinguishing a sole child as being either left or right. Figure B.7(b) shows a binary tree that differs from the tree in Figure B.7(a) because of the position of one node. Considered as ordered trees, however, the two trees are identical.

We can represent the positioning information in a binary tree by the internal nodes of an ordered tree, as shown in Figure B.7(c). The idea is to replace each missing child in the binary tree with a node having no children. These leaf nodes are drawn as squares in the figure. The tree that results is a **full binary tree**: each node is either a leaf or has degree exactly 2. There are no degree-1 nodes. Consequently, the order of the children of a node preserves the position information.

We can extend the positioning information that distinguishes binary trees from ordered trees to trees with more than 2 children per node. In a **positional tree**, the

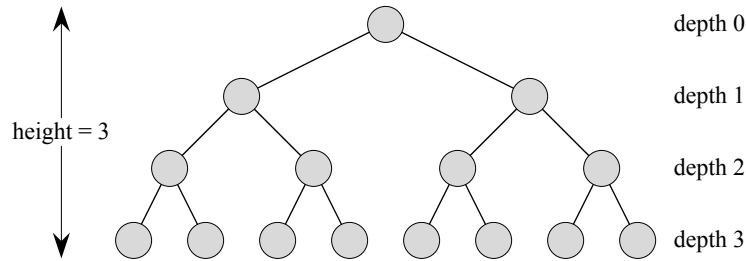


Figure B.8 A complete binary tree of height 3 with 8 leaves and 7 internal nodes.

children of a node are labeled with distinct positive integers. The i th child of a node is **absent** if no child is labeled with integer i . A **k -ary tree** is a positional tree in which for every node, all children with labels greater than k are missing. Thus, a binary tree is a k -ary tree with $k = 2$.

A **complete k -ary tree** is a k -ary tree in which all leaves have the same depth and all internal nodes have degree k . Figure B.8 shows a complete binary tree of height 3. How many leaves does a complete k -ary tree of height h have? The root has k children at depth 1, each of which has k children at depth 2, etc. Thus, the number of leaves at depth h is k^h . Consequently, the height of a complete k -ary tree with n leaves is $\log_k n$. The number of internal nodes of a complete k -ary tree of height h is

$$\begin{aligned} 1 + k + k^2 + \cdots + k^{h-1} &= \sum_{i=0}^{h-1} k^i \\ &= \frac{k^h - 1}{k - 1} \end{aligned}$$

by equation (A.5). Thus, a complete binary tree has $2^h - 1$ internal nodes.

Exercises

B.5-1

Draw all the free trees composed of the three vertices x , y , and z . Draw all the rooted trees with nodes x , y , and z with x as the root. Draw all the ordered trees with nodes x , y , and z with x as the root. Draw all the binary trees with nodes x , y , and z with x as the root.